Manual de Usuario y Administrador

Receta paso a paso para levantar y usar el pipeline RAG en AWS

Curso: Diseño de Infraestructura Escalable - BSG Institute

Proyecto 12 - LLM Data Engineering Pipeline

Autor: Emerson Olortegui

Fecha: 10/06/2026

Versión: Manual operativo final actualizado - ChromaDB persistente en S3 y redeploy ECS/ECR

Este manual está redactado como una guía práctica de réplica. La intención es que un evaluador o administrador pueda seguir los pasos en orden y obtener un resultado equivalente al ambiente construido para el proyecto.

Índice del manual

Sección	Contenido	Propósito
1	Propósito, alcance y roles	Explica qué hace la solución y quién usa cada parte.
2	Manual de usuario	Indica cómo cargar PDDs/PDFs y consultar el servicio RAG.
3	Preparación del proyecto local	Describe la estructura del repositorio, carpeta de PDDs y herramientas necesarias.
4	Preparación de AWS	Crea bucket S3, roles IAM, CloudWatch, ECS y configuración base.
5	Dockerización y publicación en ECR	Comandos para crear repositorios ECR, hacer build/tag/push de imágenes.
6	Creación de tareas ECS Fargate	Cómo crear task definitions para los microservicios batch y la API RAG.
7	Ejecución del pipeline en AWS	Orden exacto de ejecución: ETL, chunking, embeddings e indexación.
8	Levantamiento y uso de rag-api-service	Cómo levantar la API en local y en ECS Fargate.
9	Operación, validación y mantenimiento	Logs, troubleshooting, reprocesamiento, eliminación y checklist final.

1. Propósito, alcance y roles

Este manual explica cómo utilizar y levantar la solución desarrollada para el Proyecto 12. A diferencia del documento técnico, que justifica la arquitectura y las decisiones de diseño, este documento funciona como una receta operativa: primero prepara el repositorio y AWS, luego dockeriza los microservicios, publica imágenes en ECR, crea tareas ECS, ejecuta el pipeline y finalmente valida el servicio RAG.

El proyecto procesa documentos funcionales de robots RPA en formato PDF, principalmente PDDs. Estos documentos se cargan en Amazon S3 bajo raw/, luego son procesados por microservicios Python contenerizados y finalmente quedan disponibles para consultas en lenguaje natural mediante rag-api-service.

Rol	Qué puede hacer	Qué no necesita saber
Usuario funcional / evaluador	Cargar PDDs, ejecutar una consulta y validar la respuesta del RAG.	No necesita conocer Docker, ECR, ECS ni roles IAM.
Administrador técnico	Levantar la infraestructura, publicar imágenes, ejecutar tareas ECS y revisar logs.	No necesita modificar la lógica Python salvo que quiera extender el proyecto.
Profesor / revisor	Revisar evidencia de ejecución y seguir el manual para replicar el flujo.	Puede apoyarse en equivalencias AWS/Azure incluidas en el manual.

Nota para el evaluador con experiencia en Azure: en esta solución, Amazon ECR cumple un rol similar a Azure Container Registry; Amazon ECS Fargate se puede comparar conceptualmente con servicios administrados de ejecución de contenedores; Amazon S3 cumple el rol de almacenamiento de objetos; CloudWatch centraliza logs y métricas.

2. Manual de usuario

Esta sección está pensada para quien solo quiere probar el producto final. El usuario no necesita construir imágenes ni crear recursos AWS; solo debe cargar PDFs, esperar que el administrador ejecute el pipeline y consultar la API RAG.

2.1 Ubicación de los PDDs de prueba

Para facilitar la evaluación, se dejarán PDDs de prueba dentro del repositorio del proyecto compartido por Teams. La carpeta sugerida es:

```
rpa-llm-pipeline/
  documentos-busqueda/
      Formato 4.2-Generación de Cronogramas PN V.1.0.2.pdf
      otros_pdds_de_prueba.pdf
```

Estos documentos sirven como insumo para que el profesor pueda repetir la carga en S3 sin tener que solicitar archivos adicionales. Si se agregan nuevos PDDs, deben conservar extensión .pdf y un nombre legible.

2.2 Cargar PDFs al repositorio documental

Paso: Ingresar a la consola de AWS.

Paso: Abrir el servicio Amazon S3.

Paso: Ingresar al bucket del proyecto, por ejemplo rpa-llm-pipeline-bucket.

Paso: Abrir la carpeta o prefijo raw/.

Paso: Subir uno o más PDFs desde la carpeta documentos_prueba/pdds/ del repositorio.

Paso: Confirmar que el archivo aparezca dentro de raw/.

También puede hacerse por AWS CLI:

```
aws s3 cp documentos_prueba/pdds/ s3://<NOMBRE_BUCKET>/raw/ --recursive --exclude "*" --include "*.pdf"
```

2.3 Consultar el servicio RAG

Cuando el administrador haya ejecutado ETL, chunking e indexación, el usuario puede consultar rag-api-service. Este servicio recibe una pregunta, busca contexto relevante en ChromaDB y llama al modelo OpenAI configurado, por ejemplo gpt-4.1-nano.

```
curl -X POST http://<HOST_DEL_API>:8000/ask   -H "Content-Type: application/json"   -d '{"question":"¿Cuál es el objetivo del RPA Generación de Cronogramas?"}'
```

Si FastAPI expone documentación automática, también puede abrirse desde el navegador:

```
http://<HOST_DEL_API>:8000/docs
```

2.4 Preguntas sugeridas de validación

- ¿Cuál es el objetivo del robot descrito en el PDD?
- ¿Qué entradas utiliza el proceso de Generación de cronogramas?
- ¿Qué salidas genera el robot de Generación de cronogramas?
- ¿Qué excepciones de negocio o sistema se mencionan el rpa de Generación de cronogramas?
- ¿Qué pasos principales ejecuta la automatización de de Generación de cronogramas?
- ¿Qué sistemas o aplicaciones intervienen en el proceso de Generación de cronogramas?

2.5 Limitaciones para el usuario

- El sistema responde solo sobre documentos que ya fueron procesados e indexados.
- Si se sube un PDF nuevo, no aparecerá en las respuestas hasta ejecutar nuevamente el pipeline.
- Si la pregunta es muy genérica, la respuesta puede ser menos precisa.
- Si el PDF tiene mala calidad de texto o está escaneado sin OCR, la extracción puede ser limitada.
- El modelo LLM no reemplaza la revisión formal del PDD; resume información recuperada desde los chunks.

3. Preparación del proyecto local (Manual administrador)

Antes de crear recursos en AWS, el administrador debe tener listo el repositorio local. El proyecto contiene microservicios independientes, cada uno con su propio Dockerfile y requirements.txt. Esta separación permite construir y publicar una imagen por servicio.

3.1 Estructura esperada del repositorio

```
rpa-llm-pipeline/
  etl-metadata-service/
    app/
    Dockerfile
    requirements.txt
  chunking-service/
    app/
    Dockerfile
    requirements.txt
  embedding-index-service/
    app/
    Dockerfile
    requirements.txt
  rag-api-service/
    app/
    Dockerfile
    requirements.txt
  documentos_prueba/
    pdds/
      *.pdf
  docs/
    Documento_Tecnico_Proyecto12.docx
    Manual_Usuario_Administrador.docx
```

3.2 Herramientas necesarias en la máquina del administrador

Herramienta	Uso	Validación rápida
Docker Desktop	Construir imágenes Docker.	docker --version
AWS CLI	Crear recursos y publicar imágenes.	aws --version
Python 3.12	Pruebas locales de servicios.	python --version
VS Code / terminal	Editar y ejecutar comandos.	Abrir carpeta del proyecto.
Cuenta OpenAI	Consultar LLM desde rag-api-service.	Tener OPENAI_API_KEY.

```
docker --version
aws --version
python --version
aws configure
```

En aws configure se registran Access Key, Secret Access Key, región por defecto y formato de salida. Para este proyecto se recomienda región us-east-1.

4. Preparación de AWS

Esta sección describe los recursos mínimos para levantar la solución en AWS. Si el profesor viene de Azure, puede interpretar esta parte como la preparación del Resource Group, Storage Account, Container Registry, plataforma de contenedores, permisos y logs.

Concepto AWS	Equivalencia conceptual en Azure	Uso en el proyecto
Amazon S3	Azure Blob Storage	Almacena PDFs y artefactos procesados.
Amazon ECR	Azure Container Registry	Repositorio de imágenes Docker.
Amazon ECS Fargate	Azure Container Apps / ACI	Ejecuta contenedores sin administrar servidores.
IAM Roles	Managed Identity / RBAC	Permisos para que las tareas accedan a S3, ECR y logs.
CloudWatch Logs	Azure Monitor / Log Analytics	Logs de ejecución de contenedores.
Security Group	NSG / reglas de red	Permite acceso al puerto de la API.

NOTA: Los siguientes pasos tambien se puede hacer desde la misma interfaz gráfica de amazon

4.1 Definir variables base

Usaremos variables para no repetir valores. Reemplazar los placeholders por los datos reales de la cuenta AWS.

```
export AWS_REGION=us-east-1
export ACCOUNT_ID=<TU_ACCOUNT_ID>
export BUCKET_NAME=rpa-llm-pipeline-bucket-<SUFIJO_UNICO>
export CLUSTER_NAME=rpa-llm-cluster
export LOG_GROUP=/ecs/rpa-llm-pipeline
```

En Windows PowerShell se usa $env:AWS_REGION="us-east-1" en lugar de export.

4.2 Crear bucket S3 y carpetas lógicas

```
aws s3api create-bucket   --bucket $BUCKET_NAME   --region $AWS_REGION


aws s3api put-object --bucket $BUCKET_NAME --key raw/
aws s3api put-object --bucket $BUCKET_NAME --key processed/text/
aws s3api put-object --bucket $BUCKET_NAME --key processed/metadata/
aws s3api put-object --bucket $BUCKET_NAME --key processed/chunks/
aws s3api put-object --bucket $BUCKET_NAME --key processed/failed/
```

En S3 las carpetas son prefijos. Crear objetos vacíos con esos nombres ayuda a visualizar la estructura desde consola.

4.3 Crear grupo de logs en CloudWatch

```
aws logs create-log-group   --log-group-name $LOG_GROUP   --region $AWS_REGION
```

4.4. Crear roles IAM principales

Se recomiendan dos roles. El primero lo usa ECS para iniciar el contenedor y descargar imágenes. El segundo lo usa el código Python dentro del contenedor para leer y escribir en S3.

Rol	Lo usa	Permisos principales
ecsTaskExecutionRole	Servicio ECS/Fargate	Pull desde ECR, escritura de logs en CloudWatch, lectura de secretos si aplica.
ecsTaskRole	Código Python de los microservicios	Lectura/escritura en S3, acceso a logs y opcionalmente Parameter Store/Secrets Manager.

Crear archivo trust-policy-ecs-tasks.json:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"Service": "ecs-tasks.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Crear roles y asociar política administrada de ejecución:

```
aws iam create-role   --role-name ecsTaskExecutionRole   --assume-role-policy-document file://trust-policy-ecs-tasks.json


aws iam attach-role-policy   --role-name ecsTaskExecutionRole   --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy


aws iam create-role   --role-name ecsTaskRole   --assume-role-policy-document file://trust-policy-ecs-tasks.json
```

Crear política mínima para S3, ajustando el nombre del bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::rpa-llm-pipeline-bucket-<SUFIJO_UNICO>"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::rpa-llm-pipeline-bucket-<SUFIJO_UNICO>/*"
    }
  ]
}
aws iam create-policy   --policy-name rpa-llm-s3-access-policy   --policy-document file://rpa-llm-s3-access-policy.json
```

```
aws iam attach-role-policy   --role-name ecsTaskRole   --policy-arn arn:aws:iam::$ACCOUNT_ID:policy/rpa-llm-s3-access-policy
```

.

4.5. Crear cluster ECS Fargate

```
aws ecs create-cluster   --cluster-name $CLUSTER_NAME   --region $AWS_REGION
```

4.6 Red y Security Group para rag-api-service

Para ejecutar tareas Fargate se necesitan subnets y security groups. En una práctica académica se puede usar la VPC por defecto. Para rag-api-service, abrir el puerto 8000 solo desde la IP del evaluador o desde un balanceador.

```
# Obtener VPC por defecto
aws ec2 describe-vpcs   --filters Name=isDefault,Values=true   --query "Vpcs[0].VpcId"   --output text

# Obtener subnets de la VPC
aws ec2 describe-subnets   --filters Name=vpc-id,Values=<VPC_ID>   --query "Subnets[*].SubnetId"   --output text

# Crear security group para la API
aws ec2 create-security-group   --group-name rpa-llm-rag-api-sg   --description "Acceso a rag-api-service" --vpc-id <VPC_ID>

# Permitir entrada al puerto 8000 desde IP controlada
aws ec2 authorize-security-group-ingress   --group-id <SECURITY_GROUP_ID>   --protocol tcp   --port 8000   --cidr <IP_EVALUADOR>/32
```

Para pruebas rápidas se podría usar 0.0.0.0/0, pero no es recomendable para un ambiente real.

5. Dockerización y publicación en Amazon ECR

Cada carpeta del repositorio contiene un Dockerfile. El administrador debe construir una imagen por servicio y publicarla en ECR. Este proceso es equivalente a preparar una imagen local y subirla a un registry para que ECS Fargate pueda descargarla.

5.1 Crear repositorios ECR

```
aws ecr create-repository --repository-name etl-metadata-service --region $AWS_REGION
aws ecr create-repository --repository-name chunking-service --region $AWS_REGION
aws ecr create-repository --repository-name embedding-index-service --region $AWS_REGION
aws ecr create-repository --repository-name rag-api-service --region $AWS_REGION
```

5.2 Autenticar Docker contra ECR

```
aws ecr get-login-password --region $AWS_REGION | docker login   --username AWS   --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
```

5.3 Construir y subir una imagen

Ejemplo para etl-metadata-service:

```
cd etl-metadata-service
```

```
docker build -t etl-metadata-service:latest .

docker tag etl-metadata-service:latest   $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/etl-metadata-service:latest

docker push   $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/etl-metadata-service:latest

cd ..
```

5.4 Publicar las cuatro imágenes

El siguiente patrón permite publicar todos los servicios desde la raíz del repositorio:

```
for SERVICE in etl-metadata-service chunking-service embedding-index-service rag-api-service; do
  cd $SERVICE
  docker build -t $SERVICE:latest .
  docker tag $SERVICE:latest     $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$SERVICE:latest
  docker push     $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$SERVICE:latest
  cd ..
done
```

En PowerShell puede ejecutarse con un foreach equivalente:

```
$services = @("etl-metadata-service", "chunking-service", "embedding-index-service", "rag-api-service")
foreach ($service in $services) {
  cd $service
  docker build -t "$service`:latest" .
  docker tag "$service`:latest" "$env:ACCOUNT_ID.dkr.ecr.$env:AWS_REGION.amazonaws.com/$service`:latest"
  docker push "$env:ACCOUNT_ID.dkr.ecr.$env:AWS_REGION.amazonaws.com/$service`:latest"
  cd ..
}
```

6. Creación de tareas ECS Fargate

Una task definition indica a ECS qué imagen ejecutar, cuánta CPU y memoria asignar, qué variables de entorno usar, qué roles aplicar y dónde enviar los logs. Se recomienda crear una task definition por microservicio.

6.1 Valores recomendados por servicio

Servicio	Tipo	CPU/Memoria sugerida	Comando esperado
etl-metadata-service	Batch	1 vCPU / 2 GB	python app/main.py
chunking-service	Batch	1 vCPU / 2 GB	python app/main.py
embedding-index-service	Batch	2 vCPU / 4 GB	python app/main.py
rag-api-service	API	2 vCPU / 4 GB	uvicorn app.main:app --host 0.0.0.0 --port 8000

6.2 Variables de entorno por servicio

Servicio	Variables principales
etl-metadata-service	AWS_REGION, S3_BUCKET
chunking-service	AWS_REGION, S3_BUCKET
embedding-index-service	AWS_REGION, S3_BUCKET
rag-api-service	OPENAI_API_KEY=xxxx, OPENAI_MODEL=gpt-4.1-nano, TOP_K=5, MAX_TOKENS=300, TEMPERATURE=0, CHROMA_DB_PATH=/app/temp/chroma_db, CHROMA_COLLECTION=rpa_documents

6.3 Plantilla simplificada de task definition

La siguiente plantilla muestra la estructura general. Para cada servicio se reemplaza family, image, name, command y variables de entorno.

```
{
  "family": "etl-metadata-service-task",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "executionRoleArn": "arn:aws:iam::<ACCOUNT_ID>:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::<ACCOUNT_ID>:role/ecsTaskRole",
  "containerDefinitions": [
    {
      "name": "etl-metadata-service",
      "image": "<ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/etl-metadata-service:latest",
      "essential": true,
      "environment": [
        {"name": "AWS_REGION", "value": "us-east-1"},
        {"name": "S3_BUCKET", "value": "<BUCKET_NAME>"},
        {"name": "INPUT_PREFIX", "value": "raw/"}
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/rpa-llm-pipeline",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "etl"
        }
      }
    }
  ]
}
```

Guardar el JSON como etl-task-definition.json y registrarlo:

```
aws ecs register-task-definition   --cli-input-json file://etl-task-definition.json   --region $AWS_REGION
```

6.4 Recomendación para crear las cuatro task definitions

- etl-metadata-service-task: lee raw/ y escribe processed/text/ y processed/metadata/.
- chunking-service-task: lee processed/text/ y escribe processed/chunks/.
- embedding-index-service-task: lee processed/chunks/ y actualiza ChromaDB.
- rag-api-service-task: levanta FastAPI en el puerto 8000.

7. Ejecución del pipeline en AWS

El pipeline batch se ejecuta en orden. Cada etapa depende de la salida de la anterior. Si una etapa falla, se revisan logs, se corrige y se repite desde esa etapa.

7.1 Subir PDDs al bucket S3

```
aws s3 cp documentos_prueba/pdds/ s3://$BUCKET_NAME/raw/   --recursive   --exclude "*"   --include "*.pdf"
```

7.2 Ejecutar etl-metadata-service

```
aws ecs run-task   --cluster $CLUSTER_NAME   --launch-type FARGATE   --task-definition etl-metadata-service-task   --network-configuration
"awsvpcConfiguration={subnets=[<SUBNET_ID>],securityGroups=[<SG_ID>],assignPublicIp=ENABLED}"   --region
$AWS_REGION
```

Validar salidas:

```
aws s3 ls s3://$BUCKET_NAME/processed/text/

aws s3 ls s3://$BUCKET_NAME/processed/metadata/
```

7.3 Ejecutar chunking-service

```
aws ecs run-task   --cluster $CLUSTER_NAME   --launch-type FARGATE   --task-definition chunking-service-task
--network-configuration
"awsvpcConfiguration={subnets=[<SUBNET_ID>],securityGroups=[<SG_ID>],assignPublicIp=ENABLED}"   --region
$AWS_REGION

aws s3 ls s3://$BUCKET_NAME/processed/chunks/
```

7.4 Ejecutar embedding-index-service

```
aws ecs run-task   --cluster $CLUSTER_NAME   --launch-type FARGATE   --task-definition embedding-index-service-task   --network-configuration
"awsvpcConfiguration={subnets=[<SUBNET_ID>],securityGroups=[<SG_ID>],assignPublicIp=ENABLED}"   --region
$AWS_REGION
```

Validar en CloudWatch que se hayan leído chunks y que se haya actualizado la colección ChromaDB.

7.5 Punto importante sobre ChromaDB en AWS

Para que rag-api-service consulte los embeddings generados por embedding-index-service, ambos deben usar el mismo índice ChromaDB. En la implementación final sobre ECS Fargate, el índice no se copia dentro de la imagen Docker. embedding-index-service genera temp/chroma_db y lo sube a S3 en processed/chroma_db/. Luego rag-api-service descarga ese prefijo desde S3 al iniciar y recién después abre la colección rpa_documents. Este flujo evita perder el índice cuando termina la tarea batch y evita publicar imágenes Docker pesadas con datos internos.

8. Levantamiento y uso de rag-api-service

rag-api-service es diferente a los servicios batch. ETL, chunking e indexación terminan cuando procesan los documentos; en cambio, rag-api-service debe permanecer levantado para recibir preguntas del usuario.

8.1 Levantar rag-api-service en local

```
cd rag-api-service
python -m venv .venv


# Windows PowerShell
.\.venv\Scripts\Activate.ps1


pip install -r requirements.txt


$env:OPENAI_API_KEY="<TU_API_KEY>"
$env:OPENAI_MODEL="gpt-4.1-nano"
$env:CHROMA_DB_PATH="../embedding-index-service/temp/chroma_db"
$env:CHROMA_COLLECTION="rpa_documents"


uvicorn app.main:app --host 0.0.0.0 --port 8000
```

8.2 Probar la API local

```
curl -X POST http://localhost:8000/ask   -H "Content-Type: application/json"   -d '{"question":"¿Qué excepciones maneja el proceso?"}'
```

8.3 Levantar rag-api-service en ECS Fargate

Paso: Confirmar que la imagen rag-api-service:latest fue publicada en ECR.

Paso: Crear o actualizar la task definition rag-api-service-task con puerto 8000 expuesto y con la nueva revisión cuando cambie código o variables.

Paso: Configurar OPENAI_API_KEY, OPENAI_MODEL, AWS_REGION, BUCKET_NAME, CHROMA_DB_PATH, CHROMA_COLLECTION y CHROMA_S3_PREFIX.

Paso: Crear un ECS Service, no solo una ejecución batch, para que la API quede viva.

Paso: Seleccionar subnets y security group con entrada al puerto 8000.

Paso: Asignar IP pública para una prueba simple o usar un Application Load Balancer si se desea una exposición más ordenada.

Paso: Revisar logs en CloudWatch hasta ver que Uvicorn inició correctamente.

```
aws ecs create-service   --cluster $CLUSTER_NAME   --service-name rag-api-service   --task-definition rag-api-service-task   --desired-count 1   --launch-type FARGATE   --network-configuration "awsvpcConfiguration={subnets=[<SUBNET_ID>],securityGroups=[<SG_ID_API>],assignPublicIp=ENABLED}"   --region $AWS_REGION
```

8.4 Obtener la IP pública de la tarea RAG

Desde la consola AWS se puede abrir ECS > Cluster > Service > Tasks > Networking y copiar la Public IP. Por CLI se puede consultar la interfaz de red asociada a la tarea.

```
aws ecs list-tasks   --cluster $CLUSTER_NAME   --service-name rag-api-service


aws ecs describe-tasks   --cluster $CLUSTER_NAME   --tasks <TASK_ARN>


aws ec2 describe-network-interfaces   --network-interface-ids <ENI_ID>   --query "NetworkInterfaces[0].Association.PublicIp"   --output text
```

8.5 Probar rag-api-service en AWS

```
curl -X POST http://<PUBLIC_IP>:8000/ask   -H "Content-Type: application/json"   -d '{"question":"¿Cuál es el objetivo del RPA Generación de Cronogramas?"}'
```

9.2 Troubleshooting rápido

Problema	Causa probable	Qué revisar
ECS no descarga la imagen	Rol ecsTaskExecutionRole o URI ECR incorrecta.	Permisos ECR, tag de imagen y task definition.
ETL no genera texto	Bucket/prefijo mal configurado o falta permiso S3.	S3_BUCKET, INPUT_PREFIX, ecsTaskRole y CloudWatch.
No se generan chunks	No existe processed/text/ o texto vacío.	Salida ETL y logs de chunking-service.
Indexación falla	Memoria insuficiente o ruta ChromaDB incorrecta.	CPU/memoria ECS, CHROMA_DB_PATH y logs.
API no responde	Puerto cerrado, task detenida o Uvicorn falló.	Security group, puerto 8000, logs y health.
Respuesta sin contexto	Índice vacío o colección incorrecta.	CHROMA_COLLECTION y registros indexados.
OpenAI falla	API key o modelo incorrecto.	OPENAI_API_KEY y OPENAI_MODEL=gpt-4.1-nano.

9.3 Reprocesamiento o eliminación de documentos

Si se carga una nueva versión de un PDD, se debe reprocesar el documento y actualizar ChromaDB. Eliminar solo el PDF de raw/ no elimina automáticamente los chunks ni embeddings ya indexados.

Paso: Eliminar o reemplazar el PDF en raw/.

Paso: Eliminar artefactos asociados en processed/text/, processed/metadata/ y processed/chunks/.

Paso: Eliminar o marcar inactivos los embeddings asociados al document_id en ChromaDB.

Paso: Ejecutar nuevamente ETL, chunking e indexación.

Paso: Consultar rag-api-service para validar que responde con la información actualizada.

9.4 Mejora futura: EventBridge y Step Functions

En la implementación base, el administrador puede ejecutar las tareas en secuencia. Como mejora futura, EventBridge puede programar el inicio del batch y Step Functions puede orquestar los tres pasos: etl-metadata-service, chunking-service y embedding-index-service. rag-api-service debe permanecer fuera de Step Functions porque es un servicio de consulta bajo demanda.

9.5 Checklist final de réplica

- Existe bucket S3 con raw/ y processed/.
- Los PDDs de prueba están en el repo de Teams bajo documentos_prueba/pdds/.
- Se subió al menos un PDF a S3 raw/.
- Existen cuatro repositorios ECR.
- Las cuatro imágenes Docker fueron construidas y publicadas.
- Existe cluster ECS Fargate.
- Existen task definitions para los cuatro microservicios.
- ecsTaskExecutionRole y ecsTaskRole están creados y asociados correctamente.
- etl-metadata-service genera texto y metadatos.
- chunking-service genera archivos *_chunks.json.
- embedding-index-service actualiza ChromaDB.
- rag-api-service queda levantado y responde /ask.
- CloudWatch contiene logs de ejecución.

- Step Functions queda descrito como mejora futura, no como requisito base.

10. Actualización operativa final: ChromaDB persistente en S3

Esta sección reemplaza la duda inicial sobre cómo compartir ChromaDB entre embedding-index-service y rag-api-service en ECS Fargate. En Fargate, el disco local de una tarea no queda disponible para otra tarea. Por eso, el índice ChromaDB se guarda y restaura usando S3.

1. embedding-index-service lee processed/chunks/ desde S3.
2. Genera embeddings con all-MiniLM-L6-v2 y actualiza ChromaDB en la ruta local temp/chroma_db.
3. Al terminar, sube toda la carpeta temp/chroma_db hacia s3://<BUCKET_NAME>/processed/chroma_db/.
4. rag-api-service, al iniciar, descarga processed/chroma_db/ desde S3 hacia CHROMA_DB_PATH.
5. Después de restaurar ChromaDB, rag-api-service crea el cliente ChromaDB y atiende el endpoint POST /ask.

Diagrama Mermaid actualizado para el manual:

```
flowchart TD
    A[Subir PDFs a S3 raw/] --> B[Run Task: etl-metadata-service]
    B --> C[S3 processed/text y processed/metadata]
    C --> D[Run Task: chunking-service]
    D --> E[S3 processed/chunks]
    E --> F[Run Task: embedding-index-service]
    F --> G[ChromaDB local temp/chroma_db]
    G --> H[S3 processed/chroma_db]
    H --> I[Start/Update Service: rag-api-service]
    I --> J[Descarga ChromaDB desde S3]
    J --> K[POST /ask]
    K --> L[Respuesta RAG]

    X[Docker build/tag/push] --> Y[ECR]
    Y --> Z[Task Definition nueva revisión]
    Z --> B
    Z --> D
    Z --> F
    Z --> I
```

10.1 Cambios obligatorios en rag-api-service

- Agregar en el código un método download_folder() en S3Service para descargar todos los objetos de processed/chroma_db/.
- Ejecutar restore_chromadb_from_s3() antes de instanciar ChromaService en app.main.
- Mantener temp/chroma_db en .dockerignore. No debe viajar dentro de la imagen Docker.
- Eliminar del Dockerfile cualquier línea similar a COPY temp/chroma_db /app/temp/chroma_db.
- Configurar CHROMA_DB_PATH=temp/chroma_db o /app/temp/chroma_db, según el WORKDIR del Dockerfile. Lo importante es que coincida con la ruta donde se descarga S3.

Variable	Valor recomendado
AWS_REGION	us-east-1
BUCKET_NAME	<bucket real del proyecto>
CHROMA_S3_PREFIX	processed/chroma_db
CHROMA_DB_PATH	temp/chroma_db
CHROMA_COLLECTION	rpa_documents
OPENAI_MODEL	gpt-4.1-nano
TOP_K	5
MAX_TOKENS	300
TEMPERATURE	0

10.2 Rebuild y push corregido de rag-api-service

Usar estos comandos cuando se modifique rag-api-service. Se recomienda ejecutarlos desde la carpeta del servicio.

```
cd rag-api-service

docker build -t rag-api-service:latest .

docker tag rag-api-service:latest 312605973951.dkr.ecr.us-east-1.amazonaws.com/rag-api-service:latest

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
312605973951.dkr.ecr.us-east-1.amazonaws.com

docker push 312605973951.dkr.ecr.us-east-1.amazonaws.com/rag-api-service:latest
```

Si antes se eliminó la imagen local con docker rmi rag-api-service:latest, no hay problema; docker build la vuelve a crear.

10.3 Rebuild y push corregido de embedding-index-service

Usar estos comandos cuando se modifique embedding-index-service para que suba ChromaDB a S3.

```
cd embedding-index-service

docker build -t embedding-index-service:latest .

docker tag embedding-index-service:latest 312605973951.dkr.ecr.us-east-1.amazonaws.com/embedding-index-
service:latest

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
312605973951.dkr.ecr.us-east-1.amazonaws.com

docker push 312605973951.dkr.ecr.us-east-1.amazonaws.com/embedding-index-service:latest
```

10.4 Actualizar ECS Service o task definition

Después de publicar una imagen en ECR, ECS no siempre ejecuta automáticamente el contenedor nuevo si el servicio ya estaba levantado. Para rag-api-service, lo correcto es actualizar la task definition a la nueva revisión o forzar un nuevo despliegue.

```
# Opción A: si la task definition ya apunta a la imagen correcta y solo se quiere relanzar
aws ecs update-service   --cluster rpa-llm-cluster   --service rag-api-service   --force-new-deployment
--region us-east-1

# Opción B: si cambió ambiente/comando/puerto, registrar nueva revisión y actualizar el service
aws ecs update-service   --cluster rpa-llm-cluster   --service rag-api-service   --task-definition rag-
api-service-task:3   --region us-east-1
```

No es obligatorio eliminar el ECS Service cada vez. Si solo cambió la imagen o una nueva revisión, basta actualizar el service. Para pruebas rápidas, sí se puede detener la task anterior y dejar que el service levante otra.

10.5 Orden final recomendado de ejecución

1. Subir PDFs a S3 raw/.
2. Ejecutar etl-metadata-service-task y validar processed/text/ y processed/metadata/.
3. Ejecutar chunking-service-task y validar processed/chunks/.
4. Ejecutar embedding-index-service-task y validar en CloudWatch que subió processed/chroma_db/.
5. Actualizar o reiniciar rag-api-service para que descargue la última versión de ChromaDB desde S3.
6. Probar POST /ask desde curl o /docs.

10.6 Troubleshooting agregado por la última corrección

Síntoma	Causa probable	Solución
rag-api-service responde sin contexto	No descargó processed/chroma_db/ o CHROMA_COLLECTION no coincide.	Revisar BUCKET_NAME, CHROMA_S3_PREFIX, CHROMA_DB_PATH, logs de restore y colección rpa_documents.
La imagen Docker pesa demasiado o tarda mucho en subir	Se copió temp/chroma_db dentro de la imagen.	Volver a agregar temp/chroma_db a .dockerignore y quitar COPY temp/chroma_db del Dockerfile.
ECS sigue usando código antiguo	El service no tomó la nueva imagen o sigue en una revisión anterior.	Registrar nueva revisión de task definition o ejecutar update-service --force-new-deployment.
embedding-index-service termina bien pero API no ve cambios	La API no se reinició después de subir un nuevo processed/chroma_db/.	Actualizar/reiniciar rag-api-service para que descargue nuevamente el índice.